CSA0704 – Computer Networks

AT2 Scenario-Based Assessment — Working Guide & Q1 Code

This companion guide walks through all three scenarios. Q1 includes fully working, tested Python code you can run and adapt. Q2 and Q3 require hands-on tool use (Packet Tracer, a real Wi-Fi scan) — since the assessment sheet requires a signed declaration that the submission is your own original work, those two are given as detailed step-by-step guides and templates rather than pre-filled screenshots/data, so you do the actual practical work and observations yourself.

Q1 — Error Detection & Correction: "Catch the Corrupted Message"

Concept recap (for your README, in your own words — rephrase this)

Parity bit: An extra bit added to a group of bits so the total count of 1s is always even (even parity) or always odd (odd parity). The receiver recounts the 1s; if the count no longer matches the agreed parity, at least one bit was flipped in transit. It only reliably catches an *odd* number of bit flips (1, 3, 5...) — two flips can cancel out and slip through.

Checksum: The sender adds up numeric values derived from the message (here, the ASCII code of every character) into one number and sends it along. The receiver redoes the same sum on what it received and compares. It is more sensitive to certain multi-bit and multi-character corruption than a single parity bit, though it can still miss some patterns (e.g. two errors that happen to change the sum by an equal and opposite amount).

Working Python code (tested, runs as-is)

File: `q1_error_detection.py`. It (1) converts text to binary, (2) computes an even-parity bit, (3) flips one random bit in each of 5 test messages to simulate noise, (4) checks parity and checksum on the corrupted version, and (5) prints OK / ERROR DETECTED for each method so you can compare.

```
"""
CSA0704 - AT2 Q1: Catch the Corrupted Message
Parity bit vs Checksum error detection demo.

Run: python q1_error_detection.py
"""

import random

# ---------- 1. Convert text to binary ----------
def text_to_binary(message):
    """Convert each character to its 8-bit binary representation."""
    return ''.join(format(ord(ch), '08b') for ch in message)


# ---------- 2. Parity bit ----------
def calculate_parity_bit(binary_string, parity_type='even'):
    """
    Count the number of 1-bits and return the parity bit needed
    so the TOTAL number of 1s (including the parity bit) matches
    the chosen parity type.
    """
    ones_count = binary_string.count('1')
    if parity_type == 'even':
        # total 1s (including parity bit) must be even
        return '0' if ones_count % 2 == 0 else '1'
    else:  # odd parity
        return '1' if ones_count % 2 == 0 else '0'


def check_parity(binary_string, received_parity_bit, parity_type='even'):
    """Recompute parity on the received bits and compare."""
    expected_parity_bit = calculate_parity_bit(binary_string, parity_type)
    return expected_parity_bit == received_parity_bit


# ---------- 3. Checksum (sum of character codes) ----------
def calculate_checksum(message):
    """Simple checksum: sum of ASCII codes, wrapped to 16 bits."""
    total = sum(ord(ch) for ch in message)
    return total % 65536  # keep it a fixed-size (16-bit) value


def binary_to_text(binary_string):
```

```
    """Convert an 8-bit-per-character binary string back to text."""
    chars = [binary_string[i:i + 8] for i in range(0, len(binary_string), 8)]
    return ''.join(chr(int(c, 2)) for c in chars)


# ---------- 4. Simulate noise: flip one random bit ----------
def flip_random_bit(binary_string):
    bits = list(binary_string)
    pos = random.randint(0, len(bits) - 1)
    bits[pos] = '1' if bits[pos] == '0' else '0'
    return ''.join(bits), pos


# ---------- 5. Run the experiment ----------
def run_experiment(messages, parity_type='even'):
    print(f"{'Message':<20}{'Parity Result':<20}{'Checksum Result':<20}")
    print("-" * 60)

    parity_catches = 0
    checksum_catches = 0

    for msg in messages:
        original_binary = text_to_binary(msg)
        original_parity_bit = calculate_parity_bit(original_binary, parity_type)
        original_checksum = calculate_checksum(msg)

        # Simulate transmission noise: flip one bit
        corrupted_binary, flipped_pos = flip_random_bit(original_binary)
        corrupted_msg = binary_to_text(corrupted_binary)

        # --- Parity check on the corrupted message ---
        parity_ok = check_parity(corrupted_binary, original_parity_bit, parity_type)
        parity_result = "OK" if parity_ok else "ERROR DETECTED"
        if not parity_ok:
            parity_catches += 1

        # --- Checksum check on the corrupted message ---
        corrupted_checksum = calculate_checksum(corrupted_msg)
        checksum_ok = (corrupted_checksum == original_checksum)
        checksum_result = "OK" if checksum_ok else "ERROR DETECTED"
        if not checksum_ok:
            checksum_catches += 1

        print(f"{msg:<20}{parity_result:<20}{checksum_result:<20}")

    print("-" * 60)
    total = len(messages)
    print(f"Parity detected {parity_catches}/{total} corrupted messages")
    print(f"Checksum detected {checksum_catches}/{total} corrupted messages")


if __name__ == "__main__":
    random.seed()  # remove/seed a fixed number if you want reproducible results

    test_messages = [
        "HELLO",
        "NETWORK",
        "PACKET",
        "ROUTER",
        "WIFI6"
    ]

    print("=== Even Parity + Checksum Error Detection Demo ===\n")
    run_experiment(test_messages, parity_type='even')
```

Sample run output

```
=== Even Parity + Checksum Error Detection Demo ===

Message             Parity Result       Checksum Result
------------------------------------------------------------
HELLO               ERROR DETECTED      ERROR DETECTED
```

```
NETWORK             ERROR DETECTED      ERROR DETECTED
PACKET              ERROR DETECTED      ERROR DETECTED
ROUTER              ERROR DETECTED      ERROR DETECTED
WIFI6               ERROR DETECTED      ERROR DETECTED
------------------------------------------------------------
Parity detected 5/5 corrupted messages
```

Note: because only a single bit is flipped per message here, both methods will usually catch it every time — a single flip always changes the ASCII value (so the checksum changes) and always changes the 1-count parity by one. To see a real difference between the two methods, try modifying the code to flip *two* bits in the same character sometimes: parity can miss it (two flips can restore the original 1-count) while the checksum usually still changes. Run that comparison yourself and report the real numbers you get — that comparison, in your own words, is exactly what the rubric's 'Application of Concepts' and 'Decision Making' rows are looking for.

README.md outline (write this yourself, in your own words)

- What is a parity bit? (1–2 lines, plain language)
- What is a checksum? (1–2 lines, plain language)
- How you simulated noise (random single-bit flip)
- Your actual results table (paste your program's real output)
- Which method caught more errors in your run, and why you think that happened
- One limitation of each method

LinkedIn post draft (edit before posting)

"Just wrapped up a hands-on Computer Networks assignment building parity-bit and checksum error detectors in Python ■■. Simulated bit-flip noise on test messages and compared how each method catches corruption. Great reminder of how much invisible error-checking keeps our data reliable every time we send a message! #ComputerNetworks #Networking101"

Q2 — LAN & Multiple Access: "Set Up and Test a Mini Home/Lab Network"

This one has to be built by you in Cisco Packet Tracer, since it needs your own screenshots. Here is the exact step sequence so it goes quickly.

Step-by-step build

1. Open Packet Tracer → drag 5 PCs, 1 Switch (2960), 1 Router (e.g. 1841) onto the canvas.
2. Connect each PC to the switch using a Copper Straight-Through cable (auto-selected as "Automatically Choose Connection Type" also works).
3. Connect the switch to the router's FastEthernet port with another straight-through cable.
4. Click each PC → Desktop tab → IP Configuration. Assign static IPs on the same subnet, e.g. PC1=192.168.1.11/24, PC2=192.168.1.12/24 ... PC5=192.168.1.15/24, Gateway=192.168.1.1 for all.
5. Click the Router → configure the connected interface (e.g. GigabitEthernet0/0) with IP 192.168.1.1/24 and turn it on ("no shutdown").
6. Test connectivity: on any PC, open Desktop → Command Prompt → run "ping 192.168.1.12" (or any other PC's IP). All should reply successfully.
7. Screenshot: the full topology, one successful ping output.
8. Now delete the switch, add a Hub (Generic hub in End Devices/Hubs), rewire the same 5 PCs to the hub instead, and repeat the ping test.
9. Turn on Simulation mode (bottom-right) and send a ping again — watch how the hub lights up ALL ports (broadcasts to everyone) while the switch only lit the source and destination ports. Screenshot this comparison.

Write-up prompt (write 5–6 lines yourself after you observe it)

Answer in your own words: What did the hub do differently from the switch when a ping was sent? Why does that create more collisions as more devices are added (think: shared collision domain vs. the switch's per-port dedicated paths)? Why is a switch the standard choice in real networks like an internet café today?

Deliverables checklist

- .pkt file (both switch and hub versions, or one file with both attempts)
- Topology screenshot(s)
- Successful ping screenshot(s)
- Simulation-mode screenshot showing hub flooding vs switch's targeted delivery
- Your 5–6 line write-up
- LinkedIn post: topology screenshot + 3-line takeaway

Q3 — Wireless Basics: "Survey Your Own Wi-Fi World"

This needs a real scan from wherever you are (hostel/home/campus), so the data has to come from your device. Here's the process and a ready-to-fill template.

How to scan

1. Android: install "WiFi Analyzer" (Google Play) — free, shows SSID, signal (dBm), channel, and often the 802.11 standard.
2. Windows: open Command Prompt → run "netsh wlan show networks mode=bssid" for signal % and channel; Settings → Network & Internet → Wi-Fi → click a network → Properties can show link speed/band which hints at the standard.
3. Mac: hold Option and click the Wi-Fi icon in the menu bar to see channel, RSSI (signal), and PHY mode (standard) for the connected network; use a scanner app (e.g. WiFi Explorer) for other nearby networks.
4. Record at least 5 SSIDs with: signal strength, channel, and standard if visible.

Table template — fill with your real scan data

SSID	Signal (dBm/%)	Channel	Standard (n/ac/ax)	Wi-Fi 6?

Write-up prompt (½ page, your own words after you look at your table)

What patterns did you notice in signal strength vs. distance/walls? Were multiple networks crowding the same channel (channel congestion)? If any network showed 802.11ax (Wi-Fi 6), how did it compare? Why would Wi-Fi 6 help in a crowded space like a hostel floor or classroom — think about OFDMA (serving many devices' small data needs at once instead of one at a time) and better handling of many simultaneous connections.

Deliverables checklist

- Screenshot(s) of your scanning app/tool output
- Filled comparison table (Excel/Sheets, exported alongside this)
- Your ½ page write-up
- LinkedIn post: table screenshot + plain-language Wi-Fi 6 caption for non-technical readers

Final Submission Checklist

- GitHub repo: /q1 (code + README), /q2 (.pkt + screenshots + write-up), /q3 (table + screenshots + write-up) — clearly labeled folders
- Top-level README summarizing all three tasks
- One LinkedIn post per scenario (or a combined post covering all three) with the requested hashtags
- Submit GitHub repo link + LinkedIn post link(s) as your final deliverable
- Sign the Code of Conduct declaration only once the GitHub repo actually reflects your own completed work